Supplementary Material for DSL Coverage, Search Guidance and Hypothesis Construction in Symbolic Program Synthesis for ARC-AGI-2

Joy Bose
Hexaware Technologies
Bengaluru, India
joy.bose@acm.org

Abstract

This supplement holds experiments on a fixed sample of public training tasks and on development tasks that support the diagnosis in the main paper. They are not evaluation scores.

S1 Training-Sample Experiments

These experiments use a fixed random sample of 300 public training tasks and, for development, the 700 training tasks outside it. They are not evaluation scores. We report them in an appendix because they carry the evidence for the diagnosis (diminishing returns of library growth, the choice rule, the object-relational and cell-tree ladders), not because training performance is the goal.

S1.1 DSL coverage and search cost

The 44-primitive baseline. Plain depth-2 search over 44 primitives solves 67 of the 1,000 training tasks (6.7%, interval 5.3 to 8.4). Of these, 49 need one primitive, 14 need two, and 4 need only a colour map. At least one consistent program was found for 75 tasks, so 67 of 75 (89%) of the tasks with a fitting program were also solved on the test input. On the 120 public evaluation tasks it solves none. On the random 300-task sample it solves 19 (6.3%), in line with the full-set rate.

Growing the library. Table S1 and Figure S1(a) compare the three tiers on the same 300 tasks. Plain depth-2 search over all primitives goes from 19 solved tasks (6.3%, interval 4.1 to 9.7) with 44 primitives, to 28 (9.3%, 6.5 to 13.2) with 135, to 30 (10.0%, 7.1 to 13.9) with 201. The first jump is clear: 9 tasks solved by the bigger tier and none lost ($p = 0.004$). The second adds two tasks and loses none ($p = 0.5$). From 44 to 201 the gain is 11 tasks and no losses ($p = 0.001$). This supports H1 for the tested expansions: the first step is substantial and the second small, but with only two steps the diminishing returns are suggested, not established. The gain is smaller than the one we reported on the alphabetical subset, which shows how much a subset can matter (Section 5).

The cost grows as fast as the library. Time per task rises from 0.28 s to 4.5 s to 6.9 s, roughly 16 and 25 times the 44-primitive cost, and nodes rise from 5.7 thousand to 49 and 92 thousand.

S1.2 Learned pruning

Learned pruning is evaluated in detail in Appendix S2 (proposer recall, the 44-primitive and larger libraries, kept share, and proposer training-seed variance). In short: at the same pruning budget ($k = 24$) the proposer beats random pruning in all 10 matched paired comparisons, it never beats unpruned depth-2 search while that is affordable, and retraining it with other seeds moves the 201-primitive result between 24 and 30 solved tasks. These are sample results on training tasks, so we keep the tables in the appendix.

Table S1: Random sample of 300 training tasks, identical for every row. Cells show tasks fully solved, then (mean seconds per task; thousand search nodes per task). The "original" proposers saw the inputs of all training tasks, the "disjoint" ones only those outside this sample; a dash means the setting was not run. Random pruning keeps 16 primitives for the 44 library and 24 for the others; the cell shows the mean of five seeds and the range in brackets. Only the top-24 rows have a random control at the same budget (primary comparison); the top-32 rows are exploratory and have none. Evaluation results are 0 of 120 for the thirteen settings of Section 6.2.1.

Setting	44	135	201
All, depth 2	19 (0.28; 6)	28 (4.5; 49)	30 (6.9; 92)
All, depth 3	21 (9.63; 209)	–	–
Top 16, depth 3, original	17 (0.68; 13)	–	–
Top 24, depth 3, original	18 (1.96; 41)	24 (4.8; 43)	25 (4.4; 40)
Top 24, depth 3, disjoint	–	27 (4.8; 42)	24 (4.3; 39)
Top 32, depth 3, original	–	26 (10.6; 99)	28 (9.1; 90)
Top 32, depth 3, disjoint	–	29 (10.1; 98)	27 (9.4; 88)
Random pruning, mean of 5 seeds [range]	11.0 [9–13] (0.5; 10)	12.2 [9–16] (2.6; 30)	9.4 [7–11] (2.1; 21)

S1.3 Choice among fitting programs

For every task in the sample we computed, with plain depth 2, the outputs of all fitting programs (up to 3,000 per task) on each test input. We compare our rule (shortest first, ties by search order) with picking fitting programs uniformly at random, averaged over 200 draws per test input, and with an oracle that counts a test input as solved if any fitting program is right. Table S2 gives the result, counted in test inputs. In the 201 library 33 tasks have a fitting program, and in 18 of them the fitting programs disagree on the test output, so the choice can matter in only about half of them.

With one guess the shortest-first rule solves 1.0, 1.8 and 2.3 test inputs more than random choice (for 44, 135 and 201 primitives). With the two guesses that ARC allows, the gap shrinks to 0.5, 0.5 and 1.0, and the oracle is only 0 to 1 test input higher. So the rule helps a little at the first guess, and the second guess hides most of what is left. This broadly supports H4: fitting programs generalise at high rates, and shortest-first gives only a modest advantage over random choice, so the choice among fitting programs is not where

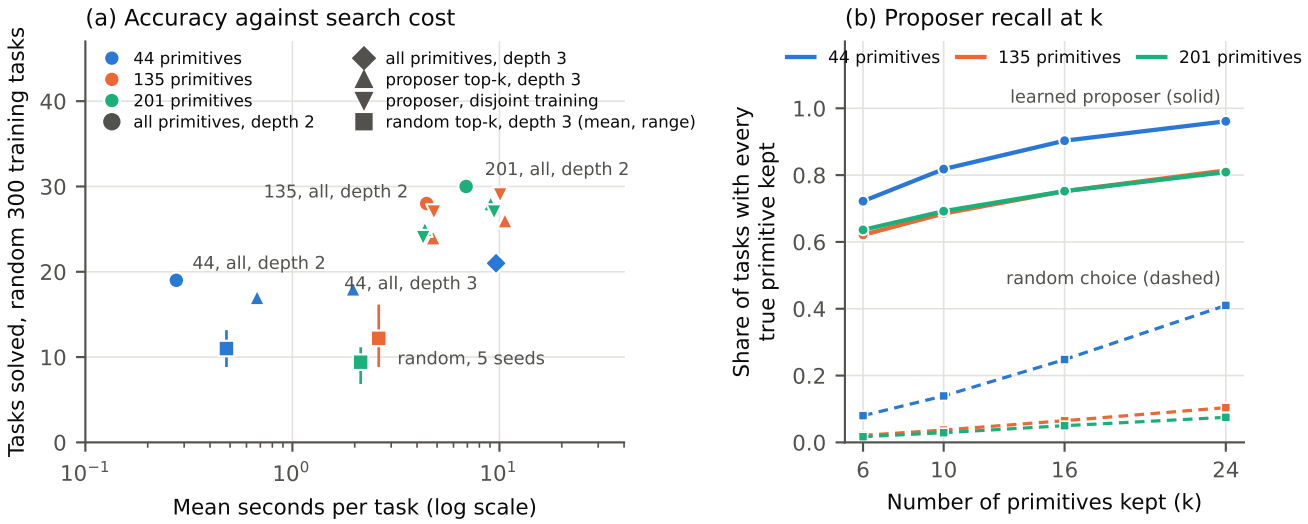


Figure S1: (a) Tasks fully solved on the random 300-task training sample against mean time per task (log scale). Random pruning is the mean of five seeds with its range as a vertical line. (b) Recall of the original proposers at k on held-out synthetic tasks, learned (solid) and random (dashed). All values are in Table S1 and Table S7.

Table S2: Choice rule against random choice, plain depth 2, 321 test inputs. Fit: tasks with a fitting program. Rival: tasks whose fitting programs give more than one distinct test output. Counts are test inputs solved; random is an average over 200 draws. The oracle solves 20, 32 and 34 test inputs for the three libraries.

Library	Fit	Rival	First guess		Two guesses	
			Short	Random	Short	Random
44	21	9	19	18.0	20	19.5
135	31	17	30	28.2	31	30.5
201	33	18	32	29.7	33	32.0

Table S3: Extensions of the object-relational solver. Development: 700 training tasks outside the sample. Sample: random 300 tasks; "fit ok" is correct fits over fits. Combined: flat solver when it finds a fit, otherwise the object-relational solver, with solved count and sign test against the flat solver alone (30). Evaluation: public evaluation tasks solved of 120.

Variant	Dev	Sample	Fit ok	Combined	Eval
v6 solver	28	12	12/18	37 ($p = 0.016$)	0
+ step 1	37	19	19/25	38 ($p = 0.008$)	0
+ step 2	39	19	19/25	38 ($p = 0.008$)	0
+ step 3	39	21	21/27	39 ($p = 0.004$)	0

the method mainly loses. It does not show that the weight 0.25 is good, because we did not vary it. In all tiers the second guess adds at most one task over the first (29 against 30 fully solved tasks for 201 primitives).

S1.3.1 Extending the object-relational solver. The object-relational solver is a menu of actions, so we asked whether a richer menu changes the evaluation result. We extended it in three steps, each developed on the 700 training tasks outside the sample and then frozen. *Step 1* adds actions that copy an object by a vector, pull an object toward another until they touch, stamp a template object at single marker pixels, and a second solver for tasks whose output is smaller than the input, which crops the object picked out by a relational test (with optional flips and rotations). *Step 2* adds relational placement: align an object with another (nearest, largest, smallest or of a given colour), put it next to that object, or mirror it across it. *Step 3* adds two-stage rules found through object correspondence: input objects are matched one to one with output objects of the same shape, which defines an intermediate grid (objects moved but not recoloured, or recoloured but not moved), and a rule set is learned for each stage.

Table S3 shows the result. Step 1 adds 9 development tasks, 7 of them crop tasks, and all 7 crop fits were correct. Step 2 adds 2 more development tasks and nothing on the sample, so we count it as a null result. Step 3 finds two extra fits on the development tasks, both wrong, and 2 more sample tasks. On the 120 public evaluation tasks none of the four variants solves anything, and all of them find the same single fit, which is wrong. The sample gain of the last variant was observed after choosing it for the combined solver, so it should be read as exploratory. The pattern repeats the one seen with the flat library: each richer menu adds a few training tasks and no evaluation tasks.

S1.3.2 Learning the rule structure. The cell-tree solver of Section 4.7 learns the structure of its rule for each task. We developed it in two rounds on the 700 development tasks (47, then 50 solved,

Table S4: Rule-structure learner (cell trees) and the combined solver. Solved tasks; share and 95% interval where given. Fit ok: correct fits over fits. 95% intervals on the sample: cell trees 5.7 to 12.0, cell trees 2 9.4 to 16.9, combined with cell trees 13.5 to 22.0, combined with cell trees 2 14.7 to 23.5 (percent).

	Dev (700)	Sample (300)	Eval (120)
Cell trees	50	25 (8.3%)	1
Fit ok	50/140	25/55	1/6
Flat (201)		30	0
Combined, with cell trees		52 (17.3%)	1
Combined, with cell trees 2		56 (18.7%)	1

Table S5: Extensions of the cell-tree learner. Solved tasks; fit ok: correct fits over fits where known.

	Dev (700)	Sample (300)	Eval (120)
Cell trees	50	25	1
Cell trees 2 (primitive first)	98	38 (12.7%)	1
Fit ok	98/215	38/85	1/7
Cell trees 4 (block features)		40	1
Colour-canonical trees	51	18	1
Augmented trees	38	18	1

7.1%), froze it, and ran it once on the sample and once on the evaluation set.

Table S4 gives the numbers. On the sample the cell trees solve 25 tasks alone, in 0.8 s per task. They solve 15 tasks the flat solver misses and 19 that the last object-relational variant misses, and miss 20 and 15 that those solve, so the three solvers are strongly complementary. The combined solver, which fills the two attempts with flat programs first, then object-relational rules, then cell trees, solves 52 of 300 (17.3%), 22 more than the flat solver and none fewer ($p < 0.001$). Its cost is 12.8 s per task. On the public evaluation tasks the cell trees solve 1 of 120 (task aa4ec2a5), find fits for 6 and get 5 of them wrong, and the combined solver also solves 1 of 120 (0.8%, interval 0.1 to 4.6) in 20.3 s per task. We did not inspect any evaluation task before the run, and one solve is weak evidence.

The cell trees fit more spuriously than the flat solver: only 36% of their fits on the development tasks are correct. But the fits can be told apart by cross-validation across the training pairs. Among fits for tasks with at least three pairs, those that reproduced every held-out pair in the leave-one-pair-out check were correct 20 times out of 21, against 15 of 79 for the others (2-pair tasks, where the check is not available: 15 of 40). For none of the 90 wrong fits on the development tasks was any of the solver's candidate trees correct, so the loss is not due to ranking: when a cell tree is wrong, the other candidate trees are wrong too.

Residual rules after a primitive, and block features. Because the solve rate of every solver was low on the public evaluation set, we tried two further ways to raise it, both developed only on the development tasks. In the first (cell trees 2), the solver tries each of the 12 DSL primitives that best match the output shape and agree most with the output cell by cell, applies it, and fits a cell tree to the remaining differences. In the second (cell trees 4), the tree also sees row and column colour counts, row and column majority colours, features of multicolour objects, and features of divider-separated blocks.

Table S5 gives the numbers. Allowing a primitive before the tree nearly doubles development coverage (50 to 98; with 6 primitives instead of 12, 94) and raises the sample result from 25 to 38. Of the 38, 19 are solved by the flat solver as well, 19 only by the cell trees, and the flat solver alone solves 11 more. The variant also solves tasks whose output size differs from the input (9 of 96 such sample tasks). Cost is 4.2 s per task on the sample and 8.9 s on the evaluation set. Fitting is more spurious than before: 117 of the 215 development fits are wrong. The combined solver with this variant solves 56 of 300 (18.7%), 26 more than the flat solver and none fewer ($p < 10^{-7}$), at 17.1 s per task.

The second extension helps little. On a larger development set of 234 tasks it solved 13 against 11, with 21 of 34 fits wrong; on the sample it solved 40, gaining 4 tasks and losing 2 against cell trees 2. This sample result was seen after the variant was frozen, so it is exploratory, and we did not put it in the combined solver.

Ideas borrowed from test-time-training systems. The winning systems of 2025 owe much to augmentation at test time, so we tried two symbolic versions of it, developed on the development tasks only. *Colour-canonical trees* relabel the colours of every pair by frequency rank before fitting, so that rules whose colours differ from pair to pair can be expressed, the symbolic counterpart of colour-permutation augmentation. *Augmented trees* fit one tree to the training pairs together with their eight rotations and flips, so they can only express rotation- and flip-invariant rules. Both are limited to tasks whose output has the input's size. On the development tasks colour-canonical trees solve 51 (4 beyond cell trees 2, at 0.5 s per task) and augmented trees 38 (3 beyond, at 6.3 s per task; 28 of 66 fits wrong, a better ratio than the others); together they lift the development union from 98 to 105. On the sample, frozen before the run, colour-canonical trees solve 18 and add nothing to cell trees 2, and augmented trees solve 18 and add 3. On the evaluation set both solve the same single task as before (aa4ec2a5); colour-canonical trees find fits for 7 tasks and augmented trees for 1, which is correct. We did not add either to the combined solver. The gains are small, and the augmented variant costs 8.7 s per task on the sample and 16.7 s on the evaluation set, which would nearly double the cost of the combined solver. So the pattern repeats: ideas that help the training tasks a little leave the evaluation count at 1 of 120.

On the evaluation set neither variant of the second extension solves anything new: both solve the same single task (aa4ec2a5), cell trees 2 find fits for 7 tasks and cell trees 4 for 9. The solve rate also falls steeply with grid size. On the development tasks cell trees 2 solve 66 of 255 tasks with fewer than 100 cells, 21 of 211 with 100 to 200, 10 of 148 with 200 to 400 and 1 of 86 with 400 or more. Public evaluation grids are larger on average, which is one plausible reason why training gains do not transfer. We did not test this explanation further.

Table S6: Examples of solved training tasks (201-primitive library) with the program that gave the correct guess. Where a colour map is shown, only the entries that change a colour are listed.

Task	Cost	Chosen program
1e0a9b12	1.00	gravity_down, no colour map
1f85a75f	1.00	crop_largest (1 of 256 fits)
0c786b71	2.00	rot180, mirror_4
0c9aba6e	1.25	nor_v, then 1→8
1b2d62fb	2.25	if_unique_recolor, and_h, then 1→8 (fourth of four tied fits, used as the second guess)
3618c87e	2.75	if_largest_fillbox, shift_down, with a four-entry colour map (5 fits, all correct)

S1.4 Case studies

Table S6 lists some solved training tasks and the programs that produced the correct guess. Two of them illustrate how the choice rule behaves.

Case 1: many explanations, shortest works. Task 1f85a75f has 256 programs that fit its two training pairs, and the cost rule picks the one-step crop to the largest object, which is correct. Section S1.3 shows how often such a choice matters.

Case 2: ambiguity survives cost ranking. The last two rows are tasks that the conditional primitives unlocked at depth 2 (solved with 201 primitives, not with 135). For 3618c87e the 135-primitive library finds no fitting program, while the 201-primitive library finds five, all correct on the test input. For 1b2d62fb the 135-primitive library already finds two fitting programs, but both give the same wrong answer. With 201 primitives the three top-ranked programs give that same wrong answer, and the correct program is tied with them on cost, so it only appears as the second guess. This shows that the two-guess rule and the tie-breaking order matter at the margin.

S2 Learned Pruning in Detail

How good is the proposer? Table S7 and Figure S1(b) show recall at k on held-out synthetic tasks. For 44 primitives, keeping the top 16 (36% of the library) keeps every needed primitive on 90.3% of tasks, against 24.8% for a random choice. For 135 and 201 primitives the recall is lower (about 75% at $k = 16$) but the gap to random choice is much larger, because random choice gets worse as the library grows. The disjoint proposers, trained without any input grid from the 300-task sample, reach almost the same recall on held-out synthetic tasks.

Recall on synthetic tasks measures generator recovery (can the network name the primitives of a random program?), not task-solution prediction (does the kept set contain a program that solves a real task?), and only the second matters for real tasks. A more direct measure is whether the primitives of a program that really solves a task are kept. For each task in the random sample whose first test input is solved by some depth-2 program, we checked whether all primitives of at least one such program are among the kept ones. For the 201 library the top-24 original proposer keeps them for 23 of 31 tasks (74%), the top-32 disjoint one for 24 of 31 (77%), and random pruning of 24 for 10 of 31 (32%). For the 135 library the figures are 21 of 29 (72%) and 25 of 29 (86%) for the original top-24 and the disjoint top-32 proposer, against 13 of 29 (45%) for random pruning. So the proposer drops the needed primitives for between 14% and 28% of the tasks that depth 2 can solve, which gives a direct explanation for much of the gap to unpruned depth-2 search, although ordering and colour-map effects may also contribute.

Table S7: Recall at k on held-out synthetic tasks: share of tasks where every primitive of the true program is kept. Random choice in brackets. Rows marked D are the disjoint proposers.

Library	$k = 6$	$k = 10$	$k = 16$	$k = 24$
44	72.2 (8.0)	81.8 (13.9)	90.3 (24.8)	96.1 (41.0)
135	62.1 (2.1)	68.5 (3.7)	75.2 (6.5)	81.4 (10.4)
135 D	62.0 (2.7)	67.6 (4.7)	74.1 (7.3)	79.9 (10.7)
201	63.6 (1.7)	69.2 (2.9)	75.2 (5.0)	80.9 (7.5)
201 D	65.1 (1.9)	70.7 (3.1)	75.8 (5.1)	81.0 (7.7)

Table S8: 44 primitives, all 1,000 training tasks (and 120 evaluation tasks), original proposer, one random seed. Share is fully solved tasks divided by all tasks.

Setting	Solved	Share	s/task	Eval
All primitives, depth 2	67	6.7%	0.28	0
Proposer top 16, depth 2	56	5.6%	0.06	0
Proposer top 16, depth 3	60	6.0%	0.70	0
Proposer top 24, depth 3	66	6.6%	1.94	0
Random 16, depth 3	39	3.9%	0.48	0

Pruning with the 44-primitive library. Table S8 shows pruning with the 44-primitive library on all 1,000 training tasks. Keeping the top 16 primitives at depth 2 is about 4.7 times faster than keeping all 44, but loses 11 tasks and gains none ($p = 0.001$). At depth 3, keeping the top 24 solves 66 tasks, almost as many as full depth 2 (67), but is slower. The proposer is clearly better than random pruning at the same budget: top 16 at depth 3 solves 60 tasks against 39 (28 tasks only the proposer solves, 7 only the random one, $p < 0.001$; one random seed).

On the random 300-task sample (Table S1), unpruned depth 3 over 44 primitives solves 21 tasks in 9.6 s each, against 19 for depth 2 in 0.28 s: two more tasks (not significant, $p = 0.5$) for 35 times the time and 37 times the nodes. Top-24 pruning at depth 3 solves 18 in 2.0 s and top-16 solves 17 in 0.68 s. So pruning does what it is meant to do for deep search, but deep search over this library is not worth much.

Does the proposer pay off with larger libraries? With 135 and 201 primitives plain depth 2 is the best or tied-best setting, and the proposer matches it only at a higher cost (H3).

- With 135 primitives, top 24 at depth 3 solves 24 (original proposer) or 27 (disjoint) tasks against 28 for plain depth 2, in about the same time (neither difference is significant: $p = 0.34$ and $p = 1.0$). Top 32 solves 26 or 29 at about 2.3 times the time of plain depth 2 ($p = 0.73$ and $p = 1.0$).

Table S9: Tasks fully solved on the random 300-task sample (depth 3 over the kept primitives) by share of the library kept. Plain depth 2 over all primitives solves 28 (135) and 30 (201).

Library	k	Kept share	Original	Disjoint
135	24	17.8%	24	27
135	32	23.7%	26	29
201	24	11.9%	25	24
201	32	15.9%	28	27

- With 201 primitives, top 24 solves 25 (original) or 24 (disjoint) against 30. The disjoint run (first seed) loses 6 tasks and gains none ($p = 0.03$), but other training seeds do better (see below), the original loses 7 and gains 2 ($p = 0.18$). Top 32 solves 28 or 27 ($p = 0.73$ and $p = 0.38$).
- Original and disjoint proposers do not differ significantly in any of the four comparisons (smallest $p = 0.25$) and the sign of the difference changes between libraries. Having seen the input grids of the 300-task sample therefore does not explain the proposer's results.
- The proposer is far better than random pruning at the same budget (H2). Random pruning with $k = 24$ solves 12.2 tasks on average with 135 primitives (range 9 to 16) and 9.4 with 201 (range 7 to 11). Comparing the disjoint proposer at $k = 24$ with each of the five random seeds gives 10 matched paired tests, our primary family. The largest p is 0.007, and in every one the proposer wins at least 11 more tasks than it loses. The proposer at $k = 32$ also beats the 24-primitive random control in all 10 comparisons (13 to 20 more tasks won than lost, every $p \leq 0.001$), but that control keeps fewer primitives, so we do not count these as matched.

Part of the loss with 201 primitives looks mechanical. A fixed k keeps a smaller share of a bigger library: $k = 24$ is 17.8% of 135 primitives but 11.9% of 201, and $k = 32$ is 23.7% and 15.9%. Table S9 lines the settings up by kept share. At similar shares (135 primitives with $k = 24$, 17.8%, and 201 primitives with $k = 32$, 15.9%) the disjoint proposer solves the same number of tasks (27 and 27) and the original one solves 24 and 28. So the 201 library does not look harder for the proposer once the budget is scaled to the library, although differences of a few tasks are within noise. This agrees with the recall figures above: for the 201 library the proposer drops the needed primitives for 23% to 26% of the solvable tasks at $k = 24$ and 32. The random control has the same property, and we kept its k fixed because the question there was a fixed candidate budget.

Proposer training seed. All proposer results above come from one training run. To measure training noise we retrained the disjoint proposers with two more network seeds and reran top-24 depth 3 on the same 300 tasks (Table S10). With 201 primitives the three seeds solve 24, 30 and 28 tasks, so the spread is as large as the gap to plain depth 2 (30): one seed matches it, and the loss we reported for the first seed is partly training noise. All six runs stay well above random pruning (means 12.2 and 9.4). This was measured only for the disjoint proposer at top-24 and depth 3. For that setting proposer training adds non-trivial variance, most of all for the 201-primitive library, and we do not claim that training is stable in general. We did not repeat the paired tests for the extra seeds and did not retrain the original proposers.

Table S10: Proposer training noise: tasks fully solved on the 300-task sample, disjoint proposer, top 24, depth 3, three network seeds. Plain depth 2 and random pruning (mean of five seeds) for reference.

Library	Seed 0	Seed 1	Seed 2	Mean	Depth 2	Random
135	27	26	26	26.3	28	12.2
201	24	30	28	27.3	30	9.4

Table S11: Tasks fully solved in the random 300-task sample (201 library, plain depth 2), by mean grid size in cells.

Mean cells	Tasks	Solved	Share
Input grid			
< 100	111	18	16%
100 to 199	98	9	9%
200 to 399	58	2	3%
≥ 400	33	1	3%
Output grid			
< 50	89	22	25%
50 to 99	48	3	6%
100 to 199	92	5	5%
≥ 200	71	0	0%

Depth 3 over the kept primitives does find a few tasks that plain depth 2 misses: 0 to 4 per setting. The union of plain depth 2 with one guided setting solves 30 to 33 tasks for the 201 library (against 30 for depth 2 alone) and 31 to 32 for the 135 library (against 28). The gain from combining is small.

S3 Further Training-Sample Material

S3.1 Solve rate and grid size

Table S11 splits the random 300-task sample (201 library, plain depth 2) by the mean number of cells in the input and output grids. The share solved falls from 16% for inputs under 100 cells to 3% for inputs over 400 cells, and no task with outputs of 200 or more cells is solved (0 of 71). In the training sample 27 of 165 tasks (16%) with outputs of at most 100 cells are solved, against 3 of 135 (2%) above that. Only 23 of the 120 evaluation tasks have outputs of at most 100 cells. As a rough calibration, applying the observed size-stratified training rates to the size distribution of the public evaluation tasks would predict about six solves, and a simple Poisson approximation gives a probability of about 0.3% for none. This is a calibration and not a formal test: tasks are not independent draws at a fixed rate given their size, and the training rates rest on few solved tasks. The calculation is descriptive and should not be read as evidence that grid size alone causes the gap to the evaluation set. So the solver shows a strong size and complexity bias, and the public evaluation tasks lie in a harder region of the observed range. Size is probably a proxy for structural complexity and not itself an explanation: a large grid with two objects can be easier than a small grid with a complicated relational rule. The comparison also ignores other differences in difficulty. We did not stratify by the number of training pairs or by the fraction of changed cells.

Table S12: Flat solver (201 primitives, plain depth 2) against the object-relational solver on the random 300-task sample, and the combination. In scope for the new solver: 204 same-size tasks. "Fit": tasks for which a rule set was found.

	Solved	Share (95% interval)	Fit
Flat, 201 primitives	30	10.0% (7.1 to 13.9)	33
Object-relational	12	4.0% (2.3 to 6.9)	18
Both solve	5		
Only object-relational	7		
Only flat	25		
Union	37	12.3% (9.1 to 16.5)	

S3.2 Object-relational solver on the sample

Table S12 shows the result. Alone, the new solver solves fewer tasks than the flat one (12 against 30; 7 tasks only it solves and 25 only the flat one solves, $p = 0.002$), even among the same-size tasks it is built for (12 against 14). Its fitting rules are right 12 times out of 18 (67%), less often than the flat solver's 91%, which is consistent with a richer language admitting more spurious fits. The richer representation raised coverage but lowered the reliability of a fit, so representational richness brings a second problem, controlling spurious explanations. It solves 7 tasks the flat solver misses. The rules include extending a line upward from objects of one colour, putting a frame around objects of given colours, sliding large objects down, connecting objects of one colour, and recolouring objects by size or parity. Using the flat solver whenever it finds a fitting program, and the new one otherwise, solves 37 tasks (12.3%): 7 more than the flat solver alone and none lost ($p = 0.016$ by the sign test).

S3.3 What depth 3 adds on training tasks

To see what depth 3 can add on training tasks, we ran the same search on 16 random sample tasks for which plain depth 2 finds no fitting program (out of 267 such tasks). One of them (0520fde7) has a fitting program at depth 3, none at depth 1 or 2, and the proposer-guided depth-3 settings do not find it. One in 16 is 6% (interval 1% to 28%), so if the rate held it would mean roughly 3 to 76 more tasks with a fitting program among the 267, and we do not know how many of those programs would be right on the test input. Depth 3 is therefore a real but probably modest source of extra coverage on training tasks, and it did not appear at all on the public evaluation tasks we tried.

S3.4 Deeper search on development tasks

The conclusion of the main paper leaves open whether depth, not representation, limits coverage. As a diagnostic (not a solver) we ran exhaustive merged-state search to depth 4 over the original 44 primitives, with one global colour map, on 100 development tasks drawn with a fixed seed before any result was seen. We did not use the evaluation tasks. Depth 5 over this library was not feasible in the time available, and the 201-primitive library was not tried at depth 4.

Depth 4 adds 3 tasks to the 11 that fit at depth 3. Of the 14 tasks with a fit, 8 needed depth 3 or 4. Depth therefore does add coverage on training tasks, slowly: about 3 tasks per 100 for each extra level here. The cost rises sharply, from 7.8 s per task at depth 3 to a mean of 51 s at depth 4 (median 44 s), with a median of about 11,000 distinct states and 1.5 million nodes. The number of fitting programs per task also grows quickly, up to 14,383 for one task at depth 4, so a fit becomes weaker evidence that the program is the right one. These numbers come from one fixed library and a sample of 100 tasks, and they say nothing about the evaluation tasks, where depth-3 search on 18 tasks found no fit.

Table S13: Development tasks (of 100) with a program that fits every training pair, by maximum depth, 44 primitives. A fit does not mean the program is right on the test input; we did not check that here.

Maximum depth	Tasks with a fit	95% interval
1	4	2 to 10
2	6	3 to 12
3	11	6 to 19
4	14	9 to 22

S4 Object Trees on Training Tasks

The object trees (main paper) solve 23 of the 700 development tasks with the first object-relational action menu and 25 with the last, against 39 for the decision-list solver of the last menu; 23 of 39 and 25 of 41 of their fits are correct. They solve 9 of the 300 sample tasks with either menu and add 2 to the 56 tasks solved by the combined solver of the main paper (flat, object-relational and residual cell trees). They take 0.4 to 0.7 s per task on the sample.

S4.1 An extended combined solver

We also tested a union of the submitted solver with three later additions: the two object trees and the colour-canonical and augmented cell trees. The additions come last and fill only attempt slots that the first three families (flat search, `orel5`, `celltree2`) leave empty, each under its own time cap, so they cannot displace an answer the submitted solver already gives. The solver was built and frozen using development and training data only. On the 300-task training sample it solves 59 tasks (19.7%, interval 15.6 to 24.5), against 56 for the submitted solver; it loses none of the 56 and gains 3. On the 120 public evaluation tasks it solves 1, the same count as before, and only 3 of those tasks used an added family. It takes 23.2 s per task on the sample (17.1 s before) and 41.7 s on the evaluation tasks (27.5 s before). We did not submit this version to Kaggle, since a gain of about one point on training tasks that does not appear on the evaluation tasks is weak evidence of a gain on the hidden test.

References